



# 设计模式

## 面向嵌入式软件开发



针对C语言及嵌入式软件开发的  
伴读教程，陪伴学习设计模式这一  
进阶知识点。

## 模板方法模式（Template Method）

### 一、意图（Intent）

定义一个操作中的算法骨架，而将某些步骤延迟到子类中，使得子类可以不改变算法结构即可重定义算法的某些步骤。

改成嵌入式 C 的话，更好理解成：

框架先把主流程固定下来，再把少数可变步骤留成钩子、回调或可替换步骤。

它控制的是：

- 流程骨架由谁决定
- 哪些步骤固定，哪些步骤允许变化
- 用户代码能插手到流程的哪个位置
- 插件 / 回调能否破坏主时序

一句话记住它：

主流程归框架，变化点归钩子。

### 二、动机（Motivation）

在嵌入式工程里，有很多流程是不能让业务层随便改顺序的：

- 网络服务接收请求的总体流程

- 配置加载后的整体生效流程
- 驱动初始化的总体阶段顺序
- 一次事务处理的固定阶段
- 一次协议包处理的解析、校验、执行、回包流程

如果不固定骨架，常见问题是：

## 1. 每个模块都自己拼流程

结果同一类流程在不同模块里顺序不同、错误处理不同、资源释放也不同。

## 2. 关键前后置动作容易漏

例如：

- 没有先校验就执行业务
- 资源没申请完就开始处理
- 出错时忘记收尾
- 某些步骤必须在统一线程上下文执行，却被乱放到别处

## 3. 业务代码容易越权改框架时序

框架本来应该保证主路径稳定，结果用户代码想怎么插就怎么插。

模板方法模式的核心价值就是：

先把不可乱动的流程骨架固定住，再开放有限的变化点。

---

## 三、适用性（Applicability）

适合使用模板方法模式的条件包括：

- 主流程必须稳定
- 不同产品或模块只在少数步骤上变化
- 某些前后置动作必须被统一保证
- 希望用户只实现钩子，不重写整个流程
- 需要框架统一控制错误处理和资源收尾

常见场景：

- 服务器 / 协议处理框架
  - 配置加载与提交流程
  - 驱动初始化框架
  - 测试步骤框架
  - 请求解析与分发框架
-

## 四、结构（Structure）

Framework Skeleton

```
|  
+-- step A (fixed)  
+-- step B (hook)  
+-- step C (fixed)  
+-- step D (hook)
```

在 C 里更常见的表现是：

固定主函数

```
|  
+-- 前置检查  
+-- 调用用户回调/钩子  
+-- 后置清理  
+-- 统一返回
```

也就是说，模板方法在嵌入式 C 中不一定依赖“继承”，更常见的是：

- 固定框架流程
- 若干可注册回调
- 若干可选钩子函数
- 严格的调用顺序

## 五、参与者（Participants）

### 1. AbstractClass / Framework Skeleton（框架骨架）

负责：

- 定义整体处理顺序
- 固定不可变步骤
- 在预定位置调用变化点
- 保证错误处理与收尾逻辑

### 2. Primitive Operations / Hooks（原语步骤 / 钩子）

这些是留给用户实现或注册的可变步骤。

在 C 中通常体现为：

- 回调函数
- ops 中的某个钩子
- 弱符号回调
- 注册表中的处理函数

### 3. Concrete Implementation（具体实现）

只负责补上变化步骤，不掌控整个框架流程。

---

## 六、协作（Collaborations）

模板方法模式的典型执行过程是：

1. 框架入口函数被调用
2. 框架先执行固定的前置步骤
3. 在指定位置调用用户钩子
4. 钩子返回后，框架继续完成后续步骤
5. 最后统一做清理、返回状态

这里最重要的是：

用户实现的是步骤，不是整个时序。

---

## 七、效果（Consequences）

### 收益

#### 1. 主流程稳定

关键时序、资源边界、收尾逻辑都由框架保证。

#### 2. 变化点集中

用户只实现被允许变化的步骤，职责边界更清楚。

#### 3. 更利于统一错误处理

前置检查、异常返回、释放资源可以收口在框架层。

#### 4. 适合做平台与业务解耦

平台层负责骨架，业务层负责少量回调实现。

### 代价

#### 1. 钩子设计不好会束手束脚

开放点太少，用户不够用；开放点太多，又会把框架打散。

#### 2. 流程调试要同时看框架和钩子

有时候逻辑问题不在钩子本身，而在钩子被调用的阶段。

### 3. 过度模板化会增加理解门槛

简单流程若硬做成模板方法，反而不如直接函数调用清楚。

---

## 八、实现要点 (Implementation)

---

### 1. 先分清“骨架步骤”和“变化步骤”

不要把所有步骤都做成钩子。真正需要固定的往往是：

- 参数检查
- 上下文建立
- 加锁 / 解锁
- 资源申请 / 释放
- 错误收尾

### 2. 钩子执行上下文必须明确

要说明：

- 钩子在哪个线程执行
- 是否允许阻塞
- 是否允许再次调用框架 API
- 回调失败后框架如何处理

### 3. 钩子返回值语义要统一

不能让每个模块自己解释成功、失败、继续还是中断。

### 4. 不要让钩子破坏骨架资源约束

例如框架持锁时调用钩子，就必须明确钩子不能做哪些动作。

---

## 九、典型应用

---

### 1. ESP-IDF HTTP Server

ESP-IDF 的 HTTP server 明确把服务器总体流程固定在框架中：`httpd_start()` 创建服务器和任务，任务循环解析请求并根据 URI 匹配已注册 handler，再由用户提供的 URI handler 处理具体业务。也就是说，请求处理骨架由框架决定，业务只填自己的处理步骤。□cite□turn262787search1□turn262787search2□turn262787search5□

### 2. 配置加载后的统一提交流程

Zephyr settings 中，加载流程会先调用各项的 `h_set`，在全部加载完成后再统一调用 `h_commit`。这正体现了“整体流程固定，局部处理留给模块自身”的模板方法思想。

### 3. 厂家 HAL 中断回调骨架

大量 MCU 厂家 SDK 都采用“固定 IRQ 处理骨架 + 用户回调”的组织方式，本质上也是模板方法，只是用 C 回调或弱符号表达，而不是类继承。

---

## 总结

---

模板方法模式在嵌入式 C 里，就是先把不可乱动的流程骨架固定住，再把少数业务差异留成受控钩子。